

Desarrollo del *inflight software* para la OBC secundaria del Satélite Quetzal-2



Autor: Flavio André Galán Donis

Curso: Diseño e Innovación

Guatemala, junio de 2026

Desarrollo del *inflight software* para la OBC secundaria del Satélite Quetzal-2

Flavio Galan

Universidad del Valle de Guatemala

gal22386@uvg.edu.gt

Resumen/Abstract

El Quetzal-2 es un nanosatélite CubeSat 2U desarrollado como proyecto académico por estudiantes y *staff* de la Universidad del Valle de Guatemala el cual pone a prueba una *On-Board Computer* (OBC) diseñada localmente que ejecuta una inteligencia artificial capaz de identificar nubes. Este nanosatélite cuenta con una computadora a bordo compuesta de por dos unidades de procesamiento, una *of-the-shelf* de GomSpace y otra desarrollada localmente utilizando Portentas H7 de Arduino. Las OBC son sistemas vitales dentro de una misión espacial puesto que gestionan todos los subsistemas del satélite así como la comunicación entre estos. Este proyecto se centra en las bases de esta OBC local a nivel de *software*, sentando las bases para los distintos modos de operación que necesita la misión, la toma de fotografías y la interoperabilidad entre las dos unidades de procesamiento que componen la OBC completa. El impacto de esta OBC desarrollada localmente representa un gran punto de inicio para próximas misiones espaciales, bajando aún más la barrera económica de entrada para desarrollar misiones espaciales con respecto a soluciones *of-the-shelf*, puesto que demuestra que no es necesario el uso de *hardware* altamente costoso con muchas más protecciones para llevar a cabo una misión exitosa.

INTRODUCCIÓN

El Quetzal-2 es un proyecto académico desarrollado por estudiantes y personal de la Universidad del Valle de Guatemala la cual pone a prueba una computadora a bordo (OBC por sus siglas en inglés) diseñada en UVG capaz de ejecutar un modelo de inteligencia artificial para identificar nubes en imágenes satelitales [1]. Este proyecto es importante no solo porque representa un avance tecnológico en Guatemala, sino también por su impacto dentro de la juventud del país, inspirando a futuros científicos, ingenieros e innovadores [2].

La misión espacial es un CubeSat 2U, 10x20x10cm y con un peso máximo de 4kg [3] y cuenta con 4 objetivos específicos que busca completar, llamadas cargas útiles (*Payloads* en inglés). Para este proyecto nos interesa específicamente el *inflight software* de la OBC secundaria de Quetzal-2 y su interacción con *Payload MILO*, el subsistema encargado del reconocimiento de nubosidad y la toma de fotografías en el espacio [1].

Las OBCs cumplen un papel clave dentro de los subsistemas del satélite puesto que son las encargadas de manejar todas las tareas, intercambio de información entre

módulos y la colecta de información sobre los demás subsistemas (*housekeeping*) antes de la conexión con la estación en tierra [4].

Algunos de los subsistemas que debe manejar una OBC son:

- Un sistema de poder (EPS por sus siglas en inglés).
- Un sistema que determina la altitud (ADCS por sus siglas en inglés).
- Un sistema de comunicación utilizando radiofrecuencias.
- Una o más cargas útiles, por ejemplo una cámara o transmisor de señales junto con su controlador.

La inicialización, intercomunicación y gestión de todos estos subsistemas recae sobre la OBC [5]. Debido a las altas limitaciones energéticas en satélites de esta escala se debe tener especial cuidado con el presupuesto de energía que se le puede brindar a cada uno de los subsistemas. De esta necesidad nacen los modos de operación, los cuáles restringen las funcionalidades del satélite según la tarea para la que se quiere que se especialice en ese momento.

Este trabajo profesional busca desarrollar un sistema de *software* para la OBC diseñada localmente (*in-house*) capaz de operar la carga útil MILO, gestionar modos de operación y ejecutar un mecanismo de *handover* con la computadora principal, validado en un entorno controlado terrestre. Para el desarrollo de este sistema se utiliza una metodología propia del laboratorio espacial UVG, basada en la metodología de ingeniería de sistemas de la NASA [6].

OBJETIVOS

Desarrollar localmente las bases de un *inflight software* para la *On Board Computer* (OBC) secundaria diseñada localmente del Quetzal-2, capaz de operar la carga útil MILO, gestionar modos de operación y ejecutar un mecanismo de *handover* con la OBC principal, validado en un entorno controlado terrestre.

1. Integrar el módulo de cámara al sistema de *software* de la computadora a bordo secundaria, logrando la captura y transmisión de imágenes desde la carga útil MILO hacia la computadora a bordo secundaria desarrollada localmente.
2. Desarrollar el sistema base de gestión de modos de operación del *software*, implementando tres modos: Arranque, Toma de fotografía y Nominal preliminar.
3. Desarrollar un Producto Mínimo Viable (MVP) del sistema de *handover* entre la OBC principal y la OBC secundaria diseñada localmente, que permita transferir y recuperar el control de la carga útil MILO.

JUSTIFICACIÓN

La misión espacial Quetzal-2 al ser una misión académica y no un proyecto de gobierno o una multinacional como Tesla, cuenta con recursos económicos relativamente limitados. Esto genera una necesidad real de reducir costos en donde sea posible, por lo tanto, un *inflight software* diseñado localmente, adaptado a las necesidades específicas de la misión pero reutilizable en misiones futuras y además es capaz de ejecutarse dentro de componentes más baratos suena muy atractivo, dando origen a este proyecto.

En cuanto a las tecnologías a utilizar, las OBCs dependen en gran medida del microcontrolador seleccionado para su misión [7]. En el Quetzal-2 se utiliza una pareja de PortentasH7 Lite para la OBC secundaria, éstas cuentan con dos procesadores ARM diseñados por el proveedor STM32 [8]. Por lo tanto, nuestras opciones se encuentran limitadas a lo que este proveedor pueda ofrecer.

Según la literatura actual, el uso de *Real Time Operating Systems* (RTOS por sus siglas en inglés) representa una práctica común para escribir sistemas complejos en el mundo del *embedded software*, incluyendo en misiones espaciales como el STUDSAT-2 [7]. Para este proyecto se utiliza FreeRTOS debido a su soporte para PortentaH7 y su comunidad de desarrolladores, la cual es grande y activa [9].

Debido a que FreeRTOS expone un API oficial en C para su desarrollo [9] y a los bajos márgenes de presupuesto de energía con los que se cuenta en misiones de escalas similares a Quetzal-2 [4], el lenguaje que se utiliza para el desarrollo del *inflight software* es C.

METODOLOGÍA

Para la documentación y organización de los módulos se utiliza una metodología propia del laboratorio pero que tiene sus bases en *Systems Engineering*, una metodología utilizada por la *National AeroSpace Agency* (NASA por sus siglas en inglés) para sus proyectos [6].

Debido al reducido tamaño del equipo de desarrollo del *inflight software* y a las prioridades cambiantes que puede tener el proyecto (por ejemplo, el cambio de protocolos) se utiliza la metodología Kanban, a continuación se presentan los 3 posibles estados en los que se puede encontrar una tarea dentro de esta metodología:

1. Por hacer: Las tareas en esta columna no se han iniciado, pero ya se encuentran definidas y listas para ser iniciadas.
2. En progreso: El desarrollo de las tareas en esta columna ya se inició, pueden o no estar bloqueadas por alguna dependencia a otra tarea en progreso o por hacer.
3. Completado: Las tareas en esta columna ya han finalizado su desarrollo, esto incluye pruebas internas en el ambiente controlado terrestre.

La modalidad de trabajo es en su mayoría presencial debido a que para realizar pruebas se necesita hardware específico que se encuentra disponible solamente dentro del laboratorio aeroespacial UVG. Sin embargo, el desarrollo de *software* tiene la gran ventaja que puede ser trabajado de forma remota también, por lo que ocasionalmente se tendrán sesiones asíncronas remotas de desarrollo.

A continuación se detallan las fases del proyecto y qué se busca completar en cada una de ellas:

1. Fase 1: Tiene como objetivo integrar la carga útil MILO con la OBC secundaria desarrollada localmente. Para lograr este objetivo se necesita establecer una comunicación básica entre ambos microcontroladores utilizando el protocolo definido por nuestro líder de módulo para luego tomar la fotografía y enviarla a las PortentasH7 Lite.
2. Fase 2: Esta fase busca implementar el sistema de gestión de modos de operación de la OBC secundaria desarrollada localmente. Por lo tanto, se necesita realizar un diseño de la arquitectura en general donde se considere definición y cambio de modos de operación, además de un camino claro para realizar pruebas de esta implementación. Luego se procede con la implementación de los distintos modos de operación validando que la toma y envío de fotografía siga funcionando.
3. Fase 3: Esta fase tiene como objetivo principal implementar el MVP del *handover* entre la OBC principal y la OBC secundaria. Para cumplir con este objetivo se necesita establecer una comunicación entre ambas OBCs utilizando el protocolo definido por nuestro líder de módulo e implementar los distintos comandos de toma y retiro de control de la OBC secundaria. Finalmente, se

necesita evaluar el correcto funcionamiento de todo el sistema con pruebas de funcionamiento.

PLAN DE TRABAJO

Tarea	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Fase 1 · Integración con la carga útil MILO																		
Pruebas de comunicación entre subsistemas																		
Definición de comandos sobre MILO																		
Desarrollo de comando de toma de fotografía																		
Desarrollo de envío de fotografía																		
Fase 2 · Sistema de gestión de modos de operación																		
Diseño de arquitectura de la OBC																		
Implementación del <i>runtime environment</i> de la OBC																		
Implementación del modo de arranque																		
Implementación del modo de toma de fotografía																		
Implementación del modo nominal preliminar																		
Fase 3 · MVP Sistema de <i>Handover</i>																		
Establecer comunicación entre OBC principal y OBC secundaria																		
Definición de comandos de comunicación																		
Implementación del comando de toma de control																		
Implementación del comando de retiro de control																		
Implementación del retiro de control de forma automática																		
Pruebas de funcionamiento y correcciones																		

BIBLIOGRAFÍA

- [1] U. del Valle de Guatemala, «Satélite CubeSat Quetzal-2». [En línea]. Disponible en: <https://www.uvg.edu.gt/quetzal-2/>
- [2] UVG, «CubeSat». [En línea]. Disponible en: <https://www.uvg.edu.gt/cubesat/>
- [3] A. Johnstone, *CubeSat Design Specification Rev. 14.1 The CubeSat Program, Cal Poly SLO CubeSat Design Specification Cal Poly -San Luis Obispo, CA Document Classification X Public Domain*. 2022. [En línea]. Disponible en: https://static1.squarespace.com/static/5418c831e4b0fa4ecac1bacd/t/62193b7fc9e72e0053f00910/1645820809779/CDS+REV14_1+2022-02-09.pdf
- [4] T. Lwabanji, R. Wilkinson, y E. Biermann Bellville, *Development of an OnBoard Computer (OBC) for a CubeSat*. 2013. [En línea]. Disponi-

ble en: https://etd.cput.ac.za/bitstream/20.500.11838/1172/1/Lumbwe_T_Final2013.pdf

- [5] S. A. Akhtar, *Design and Development of Obc of a Pico Spacecraft*. LAP Lambert Academic Publishing, 2012.
- [6] S. R. Hirshorn, *NASA Systems Engineering Handbook*. 2007. [En línea]. Disponible en: https://www.nasa.gov/wp-content/uploads/2018/09/nasa_systems_engineering_handbook_0.pdf?emrc=4096b9
- [7] B. Rajulu, S. Dasiga, y N. R. Iyer, «Open source RTOS implementation for on-board computer (OBC) in STUDSAT-2», *IEEE*, mar. 2014, doi: <https://doi.org/10.1109/aero.2014.6836377>.
- [8] Arduino, *Arduino® Portenta H7 Collective Datasheet*. 2026. [En línea]. Disponible en: <https://docs.arduino.cc/resources/datasheets/ABX00042-ABX00045-ABX00046-datasheet.pdf>
- [9] FreeRTOS, «FreeRTOS - Market Leading RTOS (Real Time Operating System) for Embedded Systems with Internet of Things Extensions». [En línea]. Disponible en: <https://www.freertos.org/>